

Facultad de Ingeniería de la Universidad de Buenos Aires



Redes de Petri

Materia: Programación Concurrente

Profesor/a: Cintia Capece

Integrantes

Alumno	Legajo	Mail
Sebastian Brizuela	105288	sbrizuela@fi.uba.ar
Raquel Ana Dávila	112002	radavila@fi.uba.ar
Lucas Facundo Couttulenc	109726	lcouttulenc@fi.uba.ar
Joel Isaac Fernandez Fox	104424	jfernandezf@fi.uba.ar
Luciano Costa	102104	lucosta@fi.uba.ar

1° Cuatrimestre 2026

ÍNDICE

Parte 1 – Fundamentos del modelado formal	3
Ejercicio 1 – ¿Por qué necesitamos Redes de Petri?	3
Ejercicio 2 – Anatomía de una Red de Petri y Semántica Dinámica	4
Parte 2 – Redes de Petri y los módulos anteriores	5
Ejercicio 3 – Exclusión mutua	5
Ejercicio 4 – Productor-Consumidor	6
Ejercicio 5 – Modelo de Actores	7
Ejercicio 6 – Two-Phase Commit (2PC)	8
Parte 3 – Verificación y análisis de la red	9
Ejercicio 7 – Análisis de Propiedades Formales	9
Ejercicio 8 – Análisis Dinámico y Traza Manual de una Red	10
Ejercicio 9 – Redes de Petri Coloreadas (CPN)	11
Parte 4 – Implementación y simulación en Rust	12
Ejercicio 10 – Motor de una Red de Petri y Exploración de Estados	12
Ejercicio 11 – Productor–Consumidor Asíncrono con Tokio	13
Ejercicio 12 – Simulación de Deadlock por Timeout (2PC)	14
Parte 5 – Análisis comparativo	15
Ejercicio 13 – Selección del Modelo de Verificación Adecuado	15

Parte 1 – Fundamentos del modelado formal

Ejercicio 1 – ¿Por qué necesitamos Redes de Petri?

Hasta el momento estudiamos diferentes mecanismos de concurrencia e implementamos soluciones sincronizadas utilizando Rust.

Respondé:

A. ¿Cuáles son las limitaciones fundamentales de verificar la correctitud de un sistema concurrente o distribuido únicamente mediante pruebas (testing empírico)?

Respuesta:

Verificar la correctitud de un sistema concurrente únicamente mediante pruebas, es decir con testing empírico, tiene varias limitaciones fundamentales que provienen de la naturaleza no determinista de este tipo de sistemas. Por ejemplo:

- Los procesos concurrentes suelen ejecutarse en infinidad de órdenes diferentes, que resulta imposible que, a medida que aumenta la cantidad de procesos, conseguir un conjunto razonable de pruebas que logren cubrir todos o la gran mayoría.
- El hecho de que una misma prueba puede arrojar resultados distintos en cada ejecución no garantiza que esta sea exitosa todo el tiempo. Que un test pase no está exento de que el sistema esté libre de fallas bajo otro orden de ejecución.
- Existen bugs que solo ocurren bajo una combinación muy específica que es muy difícil de reproducir en un entorno de prueba.
- Existencia de múltiples estados diferentes donde un conjunto de pruebas sólo puede ver una fracción muy pequeña de todos los posibles.
- Diferencias de resultados en distintos entornos. Una prueba que pueda pasar en un entorno de test puede no hacerlo en producción debido a temas como la latencia, congestión o pérdida de paquetes.
- Fallos distribuidos difíciles de simular como la caída de un nodo.
- Probar Liveness puede ser complicado debido a que no garantizas que el sistema eventualmente progresará, sino que lo hace bajo esos determinados parámetros de prueba.

En conclusión el testing empírico da evidencia parcial no implica que el sistema funcione en todas las ejecuciones posibles.

B. Explicá qué se entiende por **explosión combinatoria** del espacio de estados ($O(M^N)$).

Respuesta:

La explosión combinatoria del espacio de estados es uno de los problemas que surgen al verificar sistemas concurrentes. Describe el fenómeno por el cual el número de estados posibles de un sistema crece exponencialmente a medida que aumenta el número de componentes que lo integran.

Cuando un sistema está compuesto por N componentes y cada uno puede estar en cualquiera de los M estados posibles, el número de estados globales del sistema completo es en el peor de los casos M^N .

Esto genera una consecuencia por la cual no se pueden utilizar los tests empíricos dado que cubriría una fracción despreciable de la cantidad de estados posibles.

C. ¿Qué son los Heisenbugs y por qué aparecen frecuentemente en sistemas concurrentes?

Respuesta:

Un Heisenbug es un error de software cuyo comportamiento desaparece cuando se lo quiere observar. Son frecuentes en los sistemas concurrentes debido a que el comportamiento depende del orden exacto en el que se ejecutan los hilos o los procesos y del instante de tiempo en el que ocurre un evento.

En general aparecen en sistemas concurrentes por lo siguiente:

- El bug solo se manifiesta en un orden temporal exacto y poco probable que suele ser externo al código como que dependan del scheduler.
- La forma de debuggear cambia el timing en el que se ejecutan los procesos por lo que requieren sincronización adicional que hace que no se pueda observarlos.
- No son reproducibles a voluntad. Pueden aparecer en pocas ejecuciones sin un input claro, propio de los sistemas concurrentes.
- Al haber varios estados y variables involucradas puede aparecer un bug que cambia el valor de una variable de manera efímera que luego es difícil de reproducir.

D. Explicá cómo el modelado formal ayuda a garantizar las propiedades de Safety y Liveness antes de implementar el código.

Respuesta:

La idea central es invertir el orden habitual: en vez de escribir código y luego intentar encontrar bugs con tests, se construye primero un modelo abstracto del sistema que contenga la lógica de la concurrencia, para luego demostrar matemáticamente, sobre ese modelo, que las propiedades deseadas se cumplen antes de escribir una sola línea de código. Es útil para comprobar si es posible que pase algún error que viole la propiedad de Safety o Liveness en alguna corrida, esto hace que se pueda demostrar una garantía universal.

Para Safety la pregunta que nos hacemos al hacer el modelado formal es ¿la propia estructura del sistema hace imposible llegar a ese estado malo? y poder demostrarlo.

En cambio para Liveness la pregunta sería ¿existe alguna configuración del sistema desde la cual quedaría atrapado sin salida, para siempre? y analizando la estructura completa que genera esas ejecuciones con el modelado formal podemos también garantizar esa propiedad.

Ejercicio 2 – Anatomía de una Red de Petri y Semántica Dinámica

Considera una Red de Petri Marcada ($N = (P, T, F, W, M_0)$).

Respondé:

A. ¿Qué representa cada uno de estos elementos en el modelado de un sistema real?

- Lugar (P)
- Transición (T)
- Token / Marca
- Arco (F) y Peso (W)

Respuesta:

- Lugar (P): Indica una variable, condición, recurso o estado parcial del sistema real. Almacena la evidencia de esto mediante la presencia de tokens.
- Transición (T): Una transición representa un evento, una acción o un cambio de estado del sistema real: el envío de un mensaje, la adquisición de un lock, el procesamiento de un ítem, el disparo de una interrupción, la ejecución de una instrucción atómica.
- Token / Marca: El token es la unidad concreta que representa una instancia real de aquello que el lugar describe en abstracto: un proceso específico ocupando un estado, una unidad concreta de un recurso disponible, un mensaje concreto esperando en un buzón, un permiso concreto de acceso. La cantidad de tokens en un lugar en un momento dado es literalmente el estado del sistema en ese instante, de la misma forma en que el valor de las variables de un programa es su estado en un instante de la ejecución.
- Arco (F) y Peso (W): El arco representa la relación de causalidad o dependencia entre una condición y un evento (o entre un evento y la condición que genera): quién necesita qué para poder ocurrir, y qué produce como consecuencia. Un arco de lugar a transición modela una precondition ("para que esta acción ocurra, este recurso/condición debe estar disponible"); un arco de transición a lugar modela un efecto ("al ocurrir esta acción, se genera/libera este recurso o se cumple esta nueva condición").
El peso del arco modela la cantidad de ese recurso que la acción real consume o produce de una sola vez. Sin pesos explícitos se asume que cada acción consume/produce una única unidad discreta por cada recurso involucrado, que es el caso más común en sincronización binaria (locks, señales, flags de estado).

B. Explicá cuándo una transición t se encuentra **habilitada** e indicá la condición matemática correspondiente vista en clase:

$$M(p) \geq w(p, t) \quad \forall p \in \bullet t$$

Respuesta:

Una transición t se encuentra habilitada en un marcado M cuando todos sus lugares de entrada tienen, cada uno, una cantidad de tokens igual o mayor a lo que exige el peso del arco correspondiente. Es decir, el sistema real tiene disponibles, en ese instante, todos los recursos o condiciones necesarios para que el evento que representa t pueda ocurrir.

Para cada lugar que alimenta a la transición, debe haber al menos tantos tokens como el arco pide. Si un solo lugar de entrada no cumple esa condición (le faltan tokens), la transición no está habilitada, sin importar que el resto de sus lugares de entrada sí tengan suficientes.

$M(p)$ es la cantidad de tokens que tiene el lugar p en el marcado actual,

$W(p, t)$ es el peso del arco que va de p a t (la cantidad de tokens que ese arco exige/consumirá).

C. Describí el **disparo** de una transición utilizando la ecuación algebraica de transición de estado:

$$M' = M - Pre + Post$$

Respuesta:

La regla de disparo describe que pasa exactamente cuando un evento ocurre.

Cuando una transición habilitada se dispara, el sistema hace dos cosas a la vez, sin ningún estado intermedio visible:

1. Retira tokens de cada uno de sus lugares de entrada, en la cantidad que exige el peso de cada arco. Modela el consumo de los recursos o condiciones que ese evento necesitaba.
2. Deposita tokens en cada uno de sus lugares de salida, en la cantidad que exige el peso de cada arco. Modela la generación de nuevos recursos o condiciones como consecuencia de ese evento.

$$M' = M - Pre + Post$$

Esta fórmula captura exactamente la regla de disparo, de forma algebraica: a cada lugar se le resta lo que la transición consume de él (según el arco de entrada, si existe) y se le suma lo que la transición produce en él (según el arco de salida, si existe). Si p no es ni entrada ni salida de t , ambos términos son 0 y $M'(p) = M(p)$, entonces el lugar queda sin cambios, lo cual modela correctamente que el disparo de t es local: solo afecta a los lugares con los que tiene un arco.

D. ¿Por qué se considera que el disparo de una transición es una operación **atómica** y sin tiempo implícito?

Respuesta:

Es atómica porque Ningún otro evento del sistema puede "ver" un estado a mitad de camino del disparo. O el disparo no empezó (el marcado sigue siendo el de antes), o ya terminó completamente (el marcado es el nuevo). Esto es crucial para modelar sincronización real:

si dos procesos compiten por el mismo recurso, no existe la posibilidad de que ambos "vean" el recurso libre a la vez y ambos lo tomen, el disparo de una transición y la actualización del marcado ocurren como un único paso indivisible.

Que el disparo no tenga duración (ni tampoco una noción de "cuándo" ocurre más allá del orden causal) significa que el modelo no asume nada sobre velocidades relativas entre procesos, latencias de red, ni planificación del sistema operativo.

E. Relaciona estos elementos formales con:

- Mutex
- Semáforos
- Variables de estado / Condición de guardia

Respuesta:

Mutex

- Lugar (P): Un único lugar mutex libre representa el estado del candado: "el recurso está disponible".
- Transición (T): Adquirir el candado y liberarlo por cada proceso, lock() y unlock() respectivamente.
- Token / Marca: El único token del lugar mutex libre es literalmente el candado físico: sólo existe una copia, y quien lo posee (lo consumió) es quien tiene el derecho exclusivo de estar en la sección crítica.
- Arco (F) y Peso (W): Arco de mutex libre a adquirir el candado con peso 1: exige exactamente 1 token para poder entrar. Arco de liberar el candado a mutex libre con peso 1: al salir, se devuelve el único token. El peso fijo en 1 es lo que codifica la propiedad "binaria" del mutex nunca puede haber más de un token, así que nunca más de un proceso puede tomarlo a la vez.

Semáforos

- Lugar (P): Un lugar sem representa el semáforo mismo (el contador).
- Transición (T): $P()$ o acquire consume un token de sem. $V()$ o release produce un token en sem.
- Token / Marca: La cantidad de tokens en sem es literalmente el valor del contador del semáforo en ese instante.
- Arco (F) y Peso (W): El marcado inicial $M_0(\text{sem}) = n$ es la capacidad inicial del semáforo. Los arcos suelen tener peso 1 aunque un peso k en el arco modelaría una operación que reserva/libera k unidades de una sola vez.

Variables de estado / Condición de guardia

- Lugar (P): Un lugar puede representar directamente el valor de una variable de estado compartida
- Transición (T): La transición modela la operación que está condicionada por esa variable, el propio mecanismo de habilitación de la transición es la evaluación de la guarda.
- Token / Marca: el token representa directamente el hecho de que la condición es verdadera o una unidad discreta de una cantidad si es un contador.

- Arco (F) y Peso (W): El arco desde el lugar-variable hacia la transición, junto con su peso, es la condición de guardia en sí: $W(p,t) = k$ formaliza "esta transición requiere que la variable/condición p tenga al menos valor k para dispararse".

Parte 2 – Redes de Petri y los módulos anteriores

Ejercicio 3 – Exclusión mutua

Observá la Red de Petri correspondiente al comportamiento de un Mutex con dos procesos compitiendo (*diagrama adjunto en clase*).

Respondé:

A. ¿Qué representa al token ubicado en el lugar *Mutex*?

Respuesta:

El token en P_{mutex} representa el derecho de acceso exclusivo al recurso compartido, es la materialización, dentro del modelo, del candado disponible (el lock en estado "libre/adquirible"). Como en el marcado inicial hay exactamente un solo token en ese lugar, el modelo codifica estructuralmente que existe una sola "llave" para entrar a la sección crítica: quien la posee (quien disparó $T1_Acquire$ o $T2_Acquire$ y se llevó ese token) es, en ese instante, el único con permiso de estar dentro de $P1_CS$ o $P2_CS$. No representa una cantidad de recurso genérica, sino la posesión exclusiva de ese único recurso.

B. Explicá, **utilizando la estructura de la Red de Petri**, por qué solamente un proceso puede ingresar simultáneamente a la sección crítica.

Respuesta:

Mirando la topología del diagrama: tanto $T1_Acquire$ como $T2_Acquire$ tienen un arco de entrada compartido desde el mismo lugar P_{mutex} . Esto genera lo que en la red se llama un conflicto estructural: ambas transiciones compiten por el mismo token.

La regla de habilitación exige $M(P_{mutex}) \geq 1$ para que cualquiera de las dos pueda dispararse. Como el marcado inicial es $M(P_{mutex}) = 1$, apenas una de las dos se dispara (digamos $T1_Acquire$), el disparo consume ese único token — el marcado pasa a $M(P_{mutex}) = 0$ — y en ese nuevo marcado $T2_Acquire$ deja de estar habilitada, porque ya no hay token disponible en su precondition. $T2_Acquire$ solo vuelve a poder dispararse cuando $T1_Release$ le devuelve el token a P_{mutex} .

C. Explicá cómo la estructura de la red garantiza la propiedad de **Safety** ($M(P_{1CS}) + M(P_{2CS}) \leq 1$).

Respuesta:

Qué dice la fórmula, en primer lugar:

$M(P1_CS) + M(P2_CS) \leq 1$ dice que, sumando la cantidad de tokens que hay en el lugar $P1_CS$ más la cantidad que hay en $P2_CS$, ese total nunca puede superar 1, en ningún marcado alcanzable de la red. Como cada uno de esos lugares solo puede valer 0 o 1 (representan la condición "el proceso está en sección crítica"), que la suma sea ≤ 1 significa exactamente que nunca los dos valen 1 al mismo tiempo — es decir, nunca $P1$ y $P2$ están dentro de la sección crítica simultáneamente. Esa es la propiedad de exclusión mutua expresada como desigualdad.

Por qué la estructura de la red lo garantiza:

Una única precondition compartida. T1_Acquire y T2_Acquire no tienen cada una su propio recurso: ambas tienen un arco de entrada que sale del mismo lugar, P_mutex, que además arranca con un único token. Esto significa que ambas transiciones compiten por consumir el mismo recurso, no hay dos copias independientes, una para cada proceso.

El token se mueve, no se copia. Cuando T1_Acquire se dispara, el token de P_mutex se retira de ahí y se deposita en P1_CS, no queda ninguna copia en P_mutex. Por eso, inmediatamente después de ese disparo, P_mutex tiene 0 tokens disponibles, y T2_Acquire que también necesita un token ahí para dispararse, queda físicamente inhabilitada: no hay nada que pueda consumir.

El camino de vuelta cierra el circuito en el mismo lugar. T1_Release devuelve el token exactamente a P_mutex, de dónde había salido, no a ningún otro lugar. Esto hace que el único token exista siempre dentro de un mismo circuito cerrado ($P_mutex \leftrightarrow P1_CS$, $P_mutex \leftrightarrow P2_CS$), sin fugarse ni duplicarse en el camino.

Estas tres propiedades estructurales son precisamente las que hacen que la siguiente cantidad se mantenga constante ante cualquier disparo:

$$I(M) = M(P_mutex) + M(P1_CS) + M(P2_CS).$$

Cada transición del circuito resta 1 token de un lado y suma 1 del otro se cumple $I(M) = 1$ siempre.

Como $M(P_mutex) \geq 0$ (no puede haber una cantidad negativa de tokens), despejando de $I(M)=1$:

$$M(P1_CS) + M(P2_CS) = 1 - M(P_mutex) \leq 1$$

Queda la ecuación garantizada.

D. Relaciona esta representación formal con la semántica del tipo **Arc<Mutex<T>>** y el guardián RAII en Rust.

Respuesta:

- P_mutex: El estado "desbloqueado" del Mutex<T>
- T1/T2_Acquire: La llamada m.lock(). Si no hay token disponible en P_mutex (equivalente a que el mutex ya está tomado), el hilo se bloquea esperando, igual que lock() bloquea hasta poder tomar el candado.
- Token en P1/P2_CS: La existencia del MutexGuard devuelto por m.lock().unwrap(). Mientras ese guard está vivo, el hilo "está" en la sección crítica — igual que el token permanece en P1_CS mientras dura el acceso.
- T1/T2_Release: El drop(guard) — la devolución del token a P_mutex.
- Arc<...>: El mecanismo que permite que varios procesos/hilos (en la red: P1 y P2) tengan referencia al mismo Mutex<T>

Ejercicio 4 – Productor-Consumidor

Considera la Red de Petri del problema Productor-Consumidor con un buffer acotado de capacidad K .

Respondé:

A. Identificá en la topología de la red:

- Productor y Consumidor
- Buffer
- Espacios libres ($P_{vacíos}$)
- Espacios ocupados (P_{llenos})

Respuesta:

Productor y Consumidor: son las dos transiciones del diagrama, Producer y Consumer. Representan las acciones atómicas de "generar un dato y colocarlo en el buffer" y "tomar un dato del buffer y procesarlo", respectivamente. Son el elemento *activo* de la red los eventos que consumen y producen tokens.

Buffer: no es un único lugar, sino que está representado de forma distribuida por los dos lugares P_{llenos} (Buffer Llenos) y $P_{vacíos}$ (Espacios Vacíos). Entre los dos, en todo momento, describen completamente el estado del buffer: cuántas posiciones están ocupadas y cuántas libres.

Espacios libres ($P_{vacíos}$): el lugar cuyos tokens representan las posiciones del buffer que todavía no tienen dato es decir, la capacidad disponible en ese instante. Se inicializa con $M_0(P_{vacíos}) = K$ (toda la capacidad libre al arrancar).

Espacios ocupados (P_{llenos}): el lugar cuyos tokens representan las posiciones del buffer que ya tienen un dato esperando a ser consumido. Se inicializa con $M_0(P_{llenos}) = 0$ (buffer vacío al arrancar).

B. Explicá, cómo la red evita estructuralmente los problemas de:

- **Overflow** (producir en buffer lleno)
- **Underflow** (consumir de buffer vacío)

Respuesta:

Prevención de Overflow (producir en buffer lleno):

La transición Producer tiene, entre sus arcos de entrada, uno que sale de $P_{vacíos}$. Esto significa que producir un dato consume un token de $P_{vacíos}$ no solo deposita un token en P_{llenos} , también necesita retirar uno de $P_{vacíos}$ para poder dispararse. Cuando $P_{vacíos} = 0$ (no quedan espacios libres), la regla de habilitación $M(p) \geq W(p,t)$ falla para ese arco, y Producer queda físicamente inhabilitada: no puede dispararse, sin importar cuántos datos "quiera" producir el proceso.

Prevención de Underflow (consumir de buffer vacío):

Simétricamente, Consumer tiene como arco de entrada uno que sale de P_llenos. Consumir un dato consume un token de P_llenos, si P_llenos = 0 (no hay datos esperando), la transición Consumer no tiene qué consumir de ese lugar y queda inhabilitada.

C. ¿Qué propiedad (Invariante de Lugar) permanece constante durante toda la ejecución del sistema?

Respuesta:

La cantidad que permanece constante durante toda la ejecución es:

$$M(P_llenos) + M(P_vacíos) = K$$

Se puede verificar transición por transición:

- Productor: resta 1 a P_vacíos, suma 1 a P_llenos → suma total sin cambio.
- Consumer: resta 1 a P_llenos, suma 1 a P_vacíos → suma total sin cambio.

De este invariante se deducen directamente las dos propiedades de que no overflow ni underflow.

D. Relacioná esta red con el problema clásico estudiando en los módulos anteriores.

Respuesta:

La red modela de forma estructural el problema clásico del buffer acotado formulado por Dijkstra en 1965, también conocido como bounded buffer problem, que consiste en sincronizar un proceso productor y un proceso consumidor que comparten un buffer de capacidad limitada K, evitando que el productor escriba cuando el buffer está lleno y que el consumidor lea cuando está vacío.

De las tres restricciones que exige el problema, la red representada en el diagrama modela explícitamente dos de ellas mediante su estructura. La restricción de buffer lleno se traduce en que la transición Producer requiere consumir un token del lugar P_vacíos para poder dispararse; cuando ese lugar llega a cero, la transición queda inhabilitada de forma automática, sin que exista ningún chequeo externo agregado por el programador. Esto es la contraparte estructural exacta de la operación wait sobre un semáforo contador inicializado en la capacidad K del buffer. De manera simétrica, la restricción de buffer vacío se traduce en que Consumer requiere un token del lugar P_llenos, y cuando ese lugar está en cero, Consumer queda igualmente inhabilitada, replicando la operación wait sobre un segundo semáforo, inicializado en cero, que cuenta la cantidad de datos disponibles para consumir.

Además del consumo inicial, cada transición también produce un token en el lugar contrario al que consumió: Producer, luego de tomar un token de P_vacíos, deposita un token en P_llenos, y Consumer, luego de tomar un token de P_llenos, deposita uno en P_vacíos. Esta dinámica corresponde exactamente a la operación signal de cada semáforo sobre el otro, de modo que la red de Petri completa reproduce, con su propia semántica de disparo, la solución clásica de Dijkstra con dos semáforos contadores acoplados, sin necesidad de escribir explícitamente el código de sincronización.

Ejercicio 5 – Modelo de Actores

Observá una Red de Petri que representa dos Actores aislados comunicándose mediante un buzón de mensajes.

Respondé:

A. ¿Qué representa el lugar intermedio $P_{mailbox}$?

Respuesta:

El lugar $P_{mailbox}$ representa el medio de comunicación asincrónico entre los dos actores, funcionando como un buzón o canal intermedio que desacopla por completo la ejecución del Actor A de la ejecución del Actor B. No es un recurso que ambos actores se disputan como ocurría con el mutex, sino un canal de paso de mensajes donde el Actor A deposita lo que quiere comunicar y el Actor B lo retira cuando está en condiciones de procesarlo, sin que ambos necesiten estar sincronizados en el tiempo para que la comunicación ocurra. Es la traducción formal del principio del modelo de actores según el cual no se debe compartir memoria para comunicarse, sino comunicarse para compartir información.

B. ¿Qué representan los tokens circulando por dicho lugar?

Respuesta:

Cada token en $P_{mailbox}$ representa un mensaje enviado por el Actor A y todavía no procesado por el Actor B, es decir, una unidad de comunicación concreta que queda en tránsito hasta que el receptor la consume. La cantidad de tokens presentes en ese lugar en un instante dado equivale al tamaño de la cola de mensajes pendientes en ese momento. A diferencia del token del mutex, que representaba la posesión exclusiva de un único recurso, en este caso los tokens pueden acumularse, ya que cada uno es un mensaje independiente y no una llave compartida.

C. Explicá por qué la topología de la red demuestra que este modelo evita las **Data Races** por construcción.

Respuesta:

La razón principal es que no existe ningún lugar compartido entre las transiciones internas del Actor A y las transiciones internas del Actor B. El único punto de contacto entre ambos es $P_{mailbox}$, y ese lugar no funciona como memoria mutable compartida en el sentido clásico, sino como un canal de paso de tokens con semántica de disparo atómica. Dentro de cada actor existe un ciclo cerrado de lugares y transiciones que modela su estado interno y su procesamiento secuencial, y la única conexión hacia el exterior es una transición de envío que deposita un token en el buzón y, del otro lado, una transición de recepción que lo retira. Como el disparo de una transición es atómico, sin estados intermedios visibles, el momento en que un actor deposita el mensaje y el momento en que el otro lo toma son eventos discretos y bien definidos, y nunca existe un instante en el que ambos estén accediendo a la misma posición de memoria simultáneamente, porque estructuralmente no comparten ningún lugar donde eso pudiera ocurrir.

Además, cada actor posee exclusión mutua interna por construcción, ya que dentro de cada uno solo hay una secuencia de procesamiento activa a la vez, replicando el mismo principio de un único token circulando en un circuito cerrado que ya vimos garantiza que como máximo un proceso esté activo en cualquier instante. Esto implica que tampoco existe concurrencia interna dentro de un mismo actor que pudiera generar una condición de carrera sobre su propio estado.

En conclusión, no puede existir una data race porque no hay memoria compartida entre los actores en ningún momento. Solo existen tokens que se mueven de un lugar a otro, nunca se copian ni se acceden simultáneamente desde ambos lados, aplicando el mismo argumento estructural utilizado para demostrar exclusión mutua en el ejemplo del mutex, ahora empleado para demostrar ausencia de acceso concurrente a datos compartidos.

D. Relaciona este comportamiento con el uso de canales asincrónicos (*mpsc*) en Rust.

Respuesta:

MpMailbox representa formalmente lo que en Rust implementa un canal mpsc, compuesto por un extremo emisor y un extremo receptor conectados por una cola interna que acumula los valores enviados hasta que el receptor los procesa.

La operación de enviar un mensaje equivale a la transición de envío del Actor A. deposita un token en el canal y continúa su ejecución sin esperar a que el receptor lo procese, de la misma manera en que el disparo de una transición de salida no depende de que la transición de entrada del otro extremo se dispare en el mismo instante.

La operación de recibir un mensaje equivale a la transición de recepción del Actor B. consume un token del canal cuando hay alguno disponible y queda inhabilitada, en términos de la red, si el canal está vacío. Este es el mismo mecanismo de habilitación condicionada visto en el problema del productor consumidor, ya que un canal mpsc es estructuralmente el mismo patrón que un buffer, con la diferencia de que en Rust puede ser no acotado o acotado según el tipo de canal utilizado.

La garantía de ausencia de data races que se demuestra por la topología del grafo, al no existir ningún lugar compartido entre ambos actores salvo el canal, es la misma garantía que Rust impone en tiempo de compilación mediante el sistema de propiedad de los datos. Cuando un actor envía un valor por el canal, transfiere la propiedad de ese dato al canal y ya no puede seguir accediendo, por lo que el compilador impide que ambos actores tengan acceso simultáneo al mismo dato. Es el mismo principio estructural de la red, según el cual el token se mueve y no se copia, y solo existe en un lugar a la vez, llevado ahora al nivel del sistema de tipos, donde el compilador verifica esta propiedad automáticamente y rechaza compilar cualquier código que intente violarla.

Ejercicio 6 – Two-Phase Commit (2PC)

Considerá la Red de Petri que modela el protocolo de consenso distribuido Two-Phase Commit (2PC).

Respondé:

A. Identificá los lugares y transiciones que corresponden a:

- Coordinador
- Participantes
- Estado **Prepared**
- Decisiones **Commit** y **Abort**

Respuesta:

El Coordinador está representado por el bloque que contiene un único lugar central, junto con las transiciones ubicadas a ambos lados de ese lugar. Ese lugar central representa el estado interno del coordinador, por ejemplo esperando los votos de los participantes, y las transiciones a cada lado modelan el envío del mensaje PREPARE hacia cada participante y la recepción posterior de sus votos.

Los Participantes están representados por el bloque que contiene los lugares Initial, Voting, Committed y Aborted. Cada participante posee su propio conjunto de lugares que describen su ciclo de vida dentro del protocolo, comenzando en Initial, recibiendo el mensaje PREPARE y pasando a Voting, para luego evolucionar hacia Committed o hacia Aborted según la decisión final que reciba.

El Estado Prepared corresponde al lugar Voting de cada participante. Es el estado intermedio en el cual el participante ya recibió el PREPARE, ya emitió su voto de commit o de abort, y queda a la espera de la decisión final del coordinador.

Las Decisiones Commit y Abort corresponden a los lugares Committed y Aborted, que son los dos estados terminales alternativos a los que puede llegar un token desde Voting, según la decisión final que emita el coordinador.

B. Explicá qué ocurre con la distribución de tokens cuando el Coordinador falla (crash) tras la fase de votación.

Respuesta:

En el escenario descrito, ambos participantes envían su voto hacia el coordinador, pero el coordinador falla antes de poder procesar esos votos y emitir la decisión final.

En términos de tokens, cada participante ya disparó su transición de votación, por lo que su token quedó depositado en el lugar Voting, ya que salió de Initial pero todavía no llegó a Committed ni a Aborted, porque esa transición final depende de un mensaje del coordinador que nunca llega. Del lado del coordinador, el token que representaba el estado de espera de votos queda igualmente detenido en su lugar interno, porque el coordinador dejó de disparar transiciones al colapsar.

El resultado es que todos los tokens involucrados quedan congelados en un marcado intermedio, sin avanzar hacia ningún estado terminal, y el sistema completo queda inmovilizado en ese punto exacto de la ejecución.

C. ¿Cómo representa la Red de Petri la ocurrencia de un **Deadlock Estructural** en este escenario?

Respuesta:

El deadlock se representa por la ausencia de transiciones habilitadas salientes desde el lugar Voting. El token quedó depositado ahí, pero las únicas transiciones que podrían consumirlo, las que llevan hacia Committed o hacia Aborted, requieren como precondition un mensaje del coordinador que nunca se emite, debido a la falla de este último.

Se trata de un deadlock estructural y no de una simple lentitud temporal, ya que no es que el sistema vaya a progresar eventualmente si se espera lo suficiente, sino que dado el marcado alcanzado, con el coordinador caído y los participantes en Voting, no existe ninguna secuencia de disparos futura capaz de sacar a esos tokens de ese lugar. Las transiciones salientes de Voting quedan sin posibilidad de dispararse desde ese marcado en adelante, y la topología del grafo, combinada con la ausencia del disparo del coordinador, hace que el token quede geométricamente atrapado, sin ningún camino disponible hacia un estado terminal.

D. ¿Qué propiedad de correctitud (**Safety**) se mantiene inalterada incluso durante la falla del nodo?

Respuesta:

La propiedad de Safety que se mantiene es que resulta topológicamente imposible que un participante llegue a Committed mientras otro llega a Aborted. Es decir, aunque el sistema quede bloqueado, violando la propiedad de Liveness porque ningún participante progresa nunca hacia un estado terminal, nunca se llega a una decisión inconsistente entre los nodos. No existe ningún camino en la red, ni siquiera bajo la falla del coordinador, que permita que un token llegue a Committed en un participante y a Aborted en otro de forma simultánea o dentro de la misma ejecución.

Esto ocurre porque, tal como está construida la red, la transición hacia Committed y la transición hacia Aborted dependen ambas del mismo mensaje de decisión emitido por el coordinador, y como el coordinador falló, ninguna de las dos transiciones llega a dispararse para ningún participante. El sistema falla de forma segura, deteniéndose por completo en lugar de progresar hacia un estado incorrecto. Esta distinción entre un sistema que puede trabarse pero nunca miente sobre su estado es precisamente la limitación estructural más conocida del protocolo Two Phase Commit, que prioriza la consistencia por sobre la disponibilidad ante la falla de un único nodo coordinador.

Parte 3 – Verificación y análisis de la red

Ejercicio 7 – Análisis de Propiedades Formales

Para cada una de las siguientes 5 propiedades de análisis de Redes de Petri, indicá qué significa, qué problema detecta y un ejemplo práctico:

- **Reachability** (Alcanzabilidad)
- **Boundedness** (Limitación / Acotamiento)
- **Safety** (Seguridad / 1-Boundedness)
- **Liveness** (Vivacidad)
- **Reversibility** (Reversibilidad)

Integración de criterio de ingeniería:

Indicá cuál de estas propiedades considerarás la más crítica de verificar en cada uno de los siguientes sistemas y justificá brevemente:

- Sistema bancario
- Sistema ferroviario de interbloqueo
- Plataforma IoT masiva
- Servidor Web de alta concurrencia

Respuesta:

En primer lugar detallamos cada una de las propiedades de las Redes de Petri.

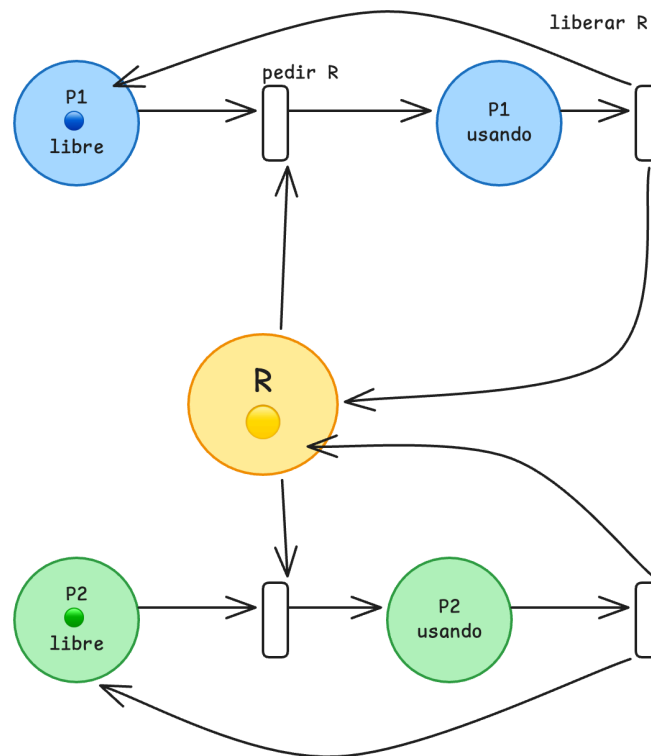
1. **Alcanzabilidad:** Se dice que un marcado M es alcanzable desde un marcado inicial M_0 si existe una secuencia de disparos de transiciones que lleva a M_0 a M . El conjunto de todos los marcados alcanzables se llama *espacio de alcanzabilidad*. Esta propiedad se utiliza para chequear si es posible llegar a estados específicos usualmente de error, o inconsistencias, de manera que se detecten estados inesperados.
2. **Limitación / Acotamiento:** Una red es acotada si el número de tokens en cualquier lugar nunca supera un valor k , para todo marcado alcanzable en otras palabras, si ese número k existe para toda la red es k -acotada. De esta forma se garantiza que los recursos modelados para bufferizar información no crezcan sin límite.
3. **Safety:** Es un caso particular de acotamiento con $k = 1$, luego, cada lugar contiene como un máximo un token en cualquier momento. Como ya hemos estudiado se garantiza la exclusión mutua entre recursos compartidos.
4. **Liveness:** Asegura que desde cualquier estado, siempre sea posible habilitar una transición en el futuro. Sirve para detectar la presencia de deadlocks y starvation.
5. **Reversibilidad:** Una red es reversible si, desde cualquier marcado alcanzable, siempre es posible regresar al marcado inicial. Esta propiedad es útil para que un sistema pueda reiniciarse o recuperarse a su estado original tras cualquier ejecución.

Ejemplo práctico:

Definimos una red para ejemplificar las propiedades, donde hay dos procesos P1 y P2 que podrían representar dos hilos de un programa, estos compiten por tener el mismo recurso

R. Cada proceso puede estar en dos estados, libre o usando el recurso, para pasar de libre a usando necesita tomar R, al terminal lo libera.

Supondremos que el marcado inicial M_0 : P1 = libre (1 token), P2 = libre (1 token), R = 1 token disponible. Todo lo demás en 0.



1. **Alcanzabilidad.** ¿Es alcanzable el marcado P1 usando + R = 0? Sí disparando pedir R desde M_0 llegamos al caso en que cuando P1 esté usando el recurso el lugar R no tenga el token, es decir no pueda ser usado.
2. **Boundedness y Safety:** Por la forma de la red nunca se puede dar el caso de que un lugar tenga más de un token, en este caso decimos que esta acotada la red con $k = 1$ lo cual garantiza el principio de Safety (exclusión mutua).
3. **Liveness:** Desde un marcado alcanzable como el caso de que P1 usando tenga el recurso, tomando el camino *liberar R*, se libera y vuelve a poner los tokens en P1 libre y R, que posteriormente puede volver al estado inicial con *pedir R*.
4. **Reversibility:** Desde un estado de P2 usando o bien P1 usando se puede volver perfectamente al marcado inicial M_0 , dicho esto se respeta esta propiedad también.

Ejercicio 8 – Análisis Dinámico y Traza Manual de una Red

Dada la Red de Petri del Productor-Consumidor vista en la diapositiva con un buffer de capacidad $K = 2$.

Respondé:

Definición de marcado $M = [P_{prod}, P_{vacios}, P_{llenos}, P_{cons}]$ donde:

- P_{prod} : Productor listo para trabajar.
- P_{vacios} : Espacios disponibles en el buffer.
- P_{llenos} : Espacios ocupados.
- P_{cons} : Consumidor listo para retirar.

A. Indicá el vector del marcado inicial M_0 .

Respuesta:

En el estado inicial, el productor y el consumidos están listos, el buffer tiene 2 espacios vacíos y no hay productos: $M_0 = [1, 2, 0, 1]$

B. Enumerá cuáles transiciones se encuentran habilitadas en el estado M_0 .

Respuesta:

1. T_{prod} (Producción/Depósito): Está habilitada, ya que requiere 1 token en P_{prod} y otro en P_{vacios} , condiciones que se cumplen en M_0 .
2. T_{cons} (Producción/Depósito): No está habilitada, ya que requiere que haya al menos un producto en el buffer.

C. Ejecutá manualmente dos disparos consecutivos a elección, mostrando el vector de marcado intermedio M_1 y el resultante M_2 .

Respuesta:

En primer lugar se hace un disparo de T_{prod} , se deposita un ítem por el productor, y luego se consume 1 token de P_{vacios} y se genera 1 token en P_{llenos} , la trama queda $M_1 = [1, 1, 1, 1]$.

En segundo lugar, producimos de nuevo ya que todavía es válido, hacemos el mismo procedimiento y el vector queda $M_2 = [1, 0, 2, 1]$.

D. ¿Qué transiciones quedan habilitadas luego de cada uno de los disparos?

Respuesta:

Luego de producir dos veces el buffer quedó completamente lleno, luego no se puede producir más pues no hay más vacíos y de esta forma se prevee el overflow por la falta de tokens en $P_{vacíos}$.

E. ¿Puede producirse un estado de Deadlock en esta red? Justificá utilizando el concepto de estado sin transiciones habilitadas.

Respuesta:

No, esta red no puede caer en deadlock, en cada uno de los tres marcados alcanzables, hay al menos una transición habilitada, esto es en el caso de producir por primera vez, en el caso donde están habilitadas ambas y en el caso final donde ya no se puede producir pero sí consumir; esto resulta en que nunca se van a bloquear dos transiciones a la vez.

Ejercicio 9 – Redes de Petri Coloreadas (CPN)

Dada la Red de Petri del Productor-Consumidor vista en la diapositiva con un buffer de capacidad $K = 2$.

Respondé:

A. ¿Qué limitación de escalabilidad presentan las Redes de Petri clásicas cuando los datos son complejos?

Respuesta:

Por un lado los tokens sin datos, impide que se modelen sistemas donde el valor de cada elemento importa, sin duplicar estructura en la red. Por otro lado la explosión de los datos, puede que trabajar con grandes cantidades de datos se vuelva inviable computacionalmente por la cantidad de verificaciones para las propiedades explicadas.

B. ¿Qué elementos nuevos incorporan las Redes de Petri Coloreadas (CPN)?

Respuesta:

Las CPN incorporan dos elementos para manejar la complejidad:

- Tipos de datos (Colores); Los tokens dejan de ser anónimos y ahora tienen un "color", lo que en términos matemáticos equivale a un valor o tipo de dato asociado.
- Lógica de Guardas: Las transiciones incluyen expresiones lógicas que evalúan los datos de los tokens antes de permitir un disparo.

C. Explicá cómo permiten modelar sistemas del mundo real sin sufrir la explosión visual del grafo.

Respuesta:

Permiten simplificar el grafo porque los tokens absorben la complejidad geométrica pues, en lugar de tener diez lugares para representar diez estados de una transacción, se utiliza un único lugar donde el "color" del token indica en qué estado se encuentra. De esta manera esto permite que el diseño sea compacto y escalable, manteniendo el rigor matemático sin saturar el diagrama

D. Relacioná los tokens coloreados y las guardas de las CPN con:

- *struct* y *enum*
- Cláusulas *match* e *if guard*
- El concepto de **Ownership** en Rust

Respuesta:

Existe un mapeo directo e isomorfismo estructural entre los elementos de una CPN y el lenguaje Rust

- **Tokens coloreados vs. struct y enum:** El "color" matemático en la red equivale exactamente al sistema de tipos riguroso de Rust. Un token puede ser una instancia de una estructura con campos de datos específicos.

- **Guardas de las CPN vs. Cláusulas *match* e *if guard*:** Las condiciones que habilitan una transición en la red se traducen directamente en el código como filtros de control de flujo (*match* con guardas) para procesar solo los datos que cumplen ciertos requisitos.
- **Tokens vs. Concepto de Ownership:** Los tokens son definidos como entidades indivisibles que representan hilos o datos. En Rust, el sistema de Ownership asegura que un dato (token) pertenezca a un solo contexto a la vez, emulando físicamente la regla de las Redes de Petri donde un token se consume de un lugar para nacer en otro, garantizando que el estado nunca se duplique ilegalmente

Parte 4 – Implementación y simulación en Rust

Ejercicio 10 – Motor de una Red de Petri y Exploración de Estados

Implementá en Rust una estructura **PetriNet** que permita:

- Almacenar el vector de marcado actual (marking).
- Representar las condiciones de entrada y salida mediante matrices de incidencia (**pre_matrix** y **post_matrix**).
- Verificar si una transición está habilitada (**is_enabled**).
- Disparar una transición de manera atómica actualizando el marcado (**fire**).
- **Explorar el espacio de estados:** Implementá un método **reachable_markings(&self) -> Vec<Vec<usize>>** que, mediante un algoritmo de búsqueda (BFS o DFS), explore y devuelva todos los marcados alcanzables desde el estado actual para una red pequeña.

```
1 use std::collections::HashSet;
2
3 struct PetriNet {
4     marking: Vec<usize>,
5     pre_matrix: Vec<Vec<usize>>, // pre_matrix[transicion][lugar]
6     post_matrix: Vec<Vec<usize>>, // post_matrix[transicion][lugar]
7 }
8
9 impl PetriNet {
10     fn new(initial_marking: Vec<usize>, pre: Vec<Vec<usize>>, post: Vec<Vec<usize>>) -> Self {
11         // ...
12     }
13
14     fn is_enabled(&self, transition: usize) -> bool {
15         // M(p) >= Pre(t, p) para todo p
16     }
17
18     fn fire(&mut self, transition: usize) -> Result<(), &'static str> {
19         // M'(p) = M(p) - Pre(t, p) + Post(t, p)
20     }
21
22     fn reachable_markings(&self) -> Vec<Vec<usize>> {
23         // Algoritmo para explorar los estados alcanzables R(M0)
24     }
25 }
```

Tarea: Utilizá tu implementación para simular la Red de Petri del Mutex (Ejercicio 3), ejecutá **reachable_markings()** y comprobá que en ningún marcado alcanzable existan dos procesos simultáneamente en sección crítica.

Respuesta:

En la implementación, la estructura **PetriNet** modela el estado dinámico del sistema a través del vector de marcado (**marking**) y las condiciones de transición mediante las matrices de incidencia (**pre_matrix** y **post_matrix**).

- **Habilitación de Transiciones (*is_enabled*):**

Verifica si una transición t está habilitada comprobando la condición $M(p) \geq Pre(t, p)$ para todos los lugares p .

```
pub fn is_enabled(&self, transition: usize) -> bool {
    if transition >= self.pre_matrix.len() {
        return false;
    }

    self.pre_matrix[transition]
        .iter()
        .zip(self.marking.iter())
        .all(|(&pre, &m)| m >= pre)
}
```

- **Disparo Atómico (*fire*):**

Actualiza el vector de marcado aplicando la ecuación de estado $M'(p) \geq M(p) - Pre(t, p) + Post(t, p)$ si la transición se encuentra habilitada.

```
pub fn fire(&mut self, transition: usize) -> Result<(), &'static str> {
    if !self.is_enabled(transition) {
        return Err("La transición no está habilitada");
    }

    for p in 0..self.marking.len() {
        self.marking[p] =
            self.marking[p] - self.pre_matrix[transition][p] +
            self.post_matrix[transition][p];
    }

    Ok(())
}
```

- **Exploración de Estados Alcanzables (*reachable_markings*):**

Aplica un algoritmo de búsqueda, para la exploración, reutiliza una instancia auxiliar *temp_net* sobre la cual evalúa *is_enabled* y *fire*, descubriendo todos los marcados en $R(M_0)$.

```
let mut temp_net = self.clone();

while let Some(current_marking) = queue.pop_front() {
    result.push(current_marking.clone());

    for t in 0..num_transitions {
        temp_net.marking = current_marking.clone();

        if temp_net.is_enabled(t) {
            if temp_net.fire(t).is_ok() {
                let next_marking = temp_net.marking().to_vec();
            }
        }
    }
}
```



```
        if !visited.contains(&next_marking) {  
            visited.insert(next_marking.clone());  
            queue.push_back(next_marking);  
        }  
    }  
}  
}
```

Ejercicio 11 – Productor–Consumidor Asincrónico con Tokio

Implementá en Rust un sistema Productor–Consumidor utilizando un canal acotado **`tokio::sync::mpsc::channel(CAPACIDAD)`**.

Analizá y respondé en los comentarios del código:

A. ¿En qué momento exacto la tarea Productora queda suspendida asincrónicamente?

Respuesta:

La tarea Productora se suspende en el instante exacto en que invoca **`tx.send(item).await`** y el canal ya contiene un número de elementos igual a **`CAPACIDAD`**.

B. ¿Qué acción del Consumidor habilita nuevamente la ejecución del Productor?

Respuesta:

La ejecución del Productor se habilita nuevamente cuando el Consumidor invoca la función **`rx.recv().await`** y remueve exitosamente un ítem del canal.

C. Explicá cómo se interpreta este comportamiento dinámico mediante el intercambio de tokens ($P_{\text{vacíos}}$ y P_{llenos}) en la Red de Petri equivalente.

Respuesta:

En una Red de Petri equivalente para un Buffer Acotado de capacidad N , el canal se modela con dos lugares principales:

- $P_{\text{vacíos}}$: Contiene fichas (tokens) que representan los espacios disponibles en el canal. Marcado inicial $M_0(P_{\text{vacíos}}) = N$.
- P_{llenos} : Contiene fichas que representan los elementos depositados listos para consumir. Marcado inicial $M_0(P_{\text{llenos}}) = 0$.

Suspensión del Productor:

Cada disparo de la transición *Enviar* toma 1 token de $P_{\text{vacíos}}$ y deposita 1 token en P_{llenos} . Cuando se han enviado N elementos sin ser consumidos, el lugar $P_{\text{vacíos}}$ se queda con 0 tokens. De esta forma, la transición *Enviar* deja de estar habilitada ($P_{\text{vacíos}} < 1$), bloqueando al Productor.

Rehabilitación del Productor:

Cuando el Consumidor dispara la transición *Recibir*, toma 1 token de P_{llenos} y deposita 1 token nuevo en $P_{\text{vacíos}}$. La llegada de esta nueva marca a $P_{\text{vacíos}}$ vuelve a cumplir la condición de disparo ($P_{\text{vacíos}} \geq 1$), rehabilitando la transición *Enviar* y permitiendo que el Productor avance.

Ejercicio 12 – Simulación de Deadlock por Timeout (2PC)

Implementá en Rust con Tokio una simulación del protocolo Two-Phase Commit donde se simule la caída del Coordinador mediante el cierre abrupto de un canal **oneshot** o un **tokio::time::timeout**.

Al ejecutar la simulación, respondé:

A. ¿En qué estado del ciclo de vida queda detenido el proceso Participante?

Respuesta:

El Participante queda detenido esperando la decisión del Coordinador.

Tras haber emitido un voto afirmativo, el Participante ingresa a un estado de incerteza en el que ya adquirió y bloqueó los recursos locales y no puede decidir por cuenta propia si confirmar o cancelar la transacción.

B. ¿Por qué la tarea no puede avanzar ni finalizar libremente?

Respuesta:

Porque el protocolo 2PC es estrictamente dependiente de una decisión global unánime.

Si el Participante decidiera hacer *Commit*, violaría la consistencia en caso de que otro participante hubiera votado *Abort*. Si decidiera hacer *Abort*, violaría la atomicidad si el Coordinador llegó a registrar un *Global Commit* antes de caer. Al perder comunicación con el Coordinador (por cierre del canal o timeout), el Participante carece de la información necesaria para resolver el estado y se ve forzado a mantener los recursos bloqueados.

C. ¿Cómo se representa este estado de bloqueo indefinido en el Grafo de Alcanzabilidad de una Red de Petri?

Respuesta:

Se representa mediante un Bloqueo Mutuo (*Deadlock*).

En el Grafo de Alcanzabilidad, este estado corresponde a un nodo terminal en el cual ninguna transición saliente está habilitada. Las fichas quedan atrapadas en los lugares que representan el estado **PREPARED** y el bloqueo de recursos local, imposibilitando alcanzar los marcados finales deseados (**COMMITTED** o **ABORTED**).

D. ¿Qué propiedad fundamental del sistema se preserva a pesar de que el proceso quedó bloqueado?

Respuesta:

Se preserva la **Consistencia**.

El protocolo 2PC sacrifica la Disponibilidad en favor de la Consistencia. Se prefiere mantener un proceso bloqueado indefinidamente antes que arriesgar una inconsistencia en los datos.

Parte 5 – Análisis comparativo

Ejercicio 13 – Selección del Modelo de Verificación Adecuado

Para cada uno de los siguientes escenarios, seleccioná la herramienta más adecuada (**FSM**, **Redes de Petri / CPN**, o **Testing Empírico**) y **justificá obligatoriamente** la decisión técnica considerando: Safety, Liveness, Concurrencia, Complejidad y Capacidad de Detección Previa.

Escenario de Software / Hardware	Modelo Seleccionado	Justificación Técnica (Obligatoria)
1. Formulario Web Secuencial (Multi-step UI)	FSM (Máquina de Estados Finitos)	Concurrencia: Nula, flujo secuencial de un único hilo Complejidad: Muy baja, espacio de estados discreto y acotado (Paso 1 -> Paso 2 -> Confirmación) Safety: Permite validar que no se salten pasos obligatorios ni se envíen datos incompletos. Liveness: Garantiza que siempre exista una transición válida para avanzar, volver o cancelar. Detección previa: Alta mediante diagramas de estado. Redes de petri sería sobre-ingeniería innecesaria y Testing empírico no previene errores de diseño de navegación antes del código.
2. Pipeline Productor–Consumidor en Memoria	Redes de Petri / CPN	Concurrencia: Alta, múltiples hilos productores y consumidores compitiendo asincrónicamente por búferes compartidos Safety: Garantiza mediante P-Invariantes el acotamiento del búfer ($M(P_llenos) \leq K$), evitando <i>Overflow</i> y <i>Underflow</i>. Liveness: Demuestra la ausencia de <i>deadlocks</i> y la no inhabilitación permanente de transiciones. Complejidad y Detección previa: CPN absorbe los tipos de datos/prioridades sin sufrir la explosión

		topológica del grafo. Permite verificar la sincronización de manera previa al código, superando al Testing Empírico que falla ante <i>Heisenbugs</i> y no-determinismo. FSM es incapaz de modelar estados concurrentes no acoplados.
3. Core de Transacciones Bancarias Concurrentes	Redes de Petri / CPN	Concurrencia: Alta, cuentas y transacciones compitiendo en paralelo por la modificación de saldos y la adquisición de <i>locks</i>. Safety: Crítica, verifica la preservación de invariantes financieros conservativos (ej. conservación del dinero, prohibición de saldos negativos o doble gasto) mediante Guardas y P-Invariantes. Liveness: Detecta posibles <i>deadlocks</i> en transferencias cruzadas simultáneas (A->B vs B->A). Complejidad y Detección Previa: CPN modela los datos (cuentas, montos, IDs) mediante marcas tipadas (<i>struct/enum</i>). El Testing Empírico es insuficiente debido a la combinatoria inabarcable de entrelazamientos y la cobertura infinitesimal frente al estado total.
4. Protocolo de Consenso Distribuido (2PC / Raft)	Redes de Petri / CPN (<i>Model Checking</i>)	Concurrencia: Extremadamente alta y distribuida, coordinación asíncrona entre nodos propensa a latencias, caídas de red y <i>crashes</i>. Safety: Garantiza consistencia estricta (ej. imposibilidad de decisiones contradictorias <i>Commit / Abort</i> en 2PC o múltiples líderes en Raft). Liveness: Identifica estados de bloqueo indefinido (<i>Blocking States /</i>

		<i>Deadlocks</i>) cuando el coordinador o los participantes caen tras la fase de votación. Complejidad y Detección Previa: Permite verificar formalmente la resiliencia del protocolo antes del despliegue en producción. El Testing Empírico no puede simular con certeza matemática todas las permutaciones de fallas de red y temporizadores; FSM colapsa al intentar modelar la sincronización distribuida sin reloj global.
5. API REST CRUD convencional sobre Base de Datos	Testing Empírico (<i>Unitario, Integración y E2E</i>)	Concurrencia: Baja a moderada en la capa de aplicación. La concurrencia pesada y la sincronización se delegan completamente al motor de la base de datos relacional (mecanismos ACID/2PL/SSI). Complejidad: Baja en términos de flujo de control concurrente; se reduce a mapeos secuenciales de peticiones HTTP a consultas SQL. Safety y Liveness: Las garantiza el motor de base de datos underlying; la API solo ejecuta lógica procedural. Detección Previa y Costo-Beneficio: Crear un modelo formal (FSM o Petri) para un CRUD tradicional representaría un costo injustificado. El Testing Empírico (pruebas de integración con DB en contenedores) brinda una excelente cobertura, bajo costo de desarrollo y rápida validación de regresiones.
6. Sistema de Interbloqueo de Control Ferroviario	Redes de Petri / CPN	Concurrencia: Alta y físicamente paralela. Múltiples trenes desplazándose de forma

		independiente por tramos de vía compartida, cambios de agujas y señales. Safety: Crítica (Riesgo de pérdidas humanas y materiales). Demuestra formalmente la propiedad de 1-Boundedness en los lugares de vía ($M(P_{\text{vía}}) \leq 1$), garantizando Exclusión Mutua estricta para evitar colisiones. Liveness: Garantiza la ausencia total de <i>deadlocks</i> (trenes bloqueados de frente en una misma vía congelando la red). Complejidad y Detección Previa: La verificación matemática formal debe ser previa al despliegue, probar empíricamente este sistema mediante testing "en vivo" es inviable por el costo destructivo de las fallas. FSM no escala ante la combinatoria de múltiples trenes interactuando de forma concurrente.
--	--	---